

Deployment and model review — 22 September 2026

This review covers the current working tree, including the uncommitted Gemini changes, both serving ONNX files, saved training configurations/history, local dataset manifests/images, tests, UI, and container setup. It is an assessment, not a deployment or source-code refactor. Existing application files, weights, and training data were not changed. Reproducible audit code and measurements are in [output/project_review_2026_09_22/](#).

Assessment: suitable for a controlled demonstration after the critical fixes; not yet ready for an unrestricted production service. The basic architecture is appropriate for quarter-turn document orientation. The main gaps are trustworthy target-domain evaluation, safe handling of uncertain documents, bounded processing, reproducible deployment, and access/cost controls. Training a much larger model is not the first priority.

What is already sound

- FastAPI, a CPU ONNX runtime, Pillow, and a PDF renderer are a reasonable small-service stack. Training dependencies are excluded from the runtime image.
- Rotation conventions are explicit: checkpoint classes are counterclockwise, API corrections are clockwise. Tests cover the mapping and pixel rotation.
- PDFs retain original text and vector content by composing page rotation, rather than replacing pages with raster images. Existing page rotations and encrypted/page-limit failures have coverage.
- Shared preprocessing, ONNX metadata, finite-logit checks, and export parity checks reduce training/serving drift.
- Manifest loading verifies hashes, rejects exact duplicates across splits, deduplicates within splits, and prevents recorded document groups crossing splits.
- Training has ImageNet initialization, modest augmentations, AdamW, cosine learning-rate scheduling, early stopping, macro-F1 checkpoint selection, AMP, and resumable optimizer/RNG state.
- Optional OCR is lazy, its intervention is recorded, and an override does not invent a calibrated confidence score. Gemini responses accept only a constrained angle; provider errors are sanitized.
- The Docker image runs as a non-root user. `.env` is excluded from Git and the Docker build context.

The two models and the meaning of hybrid

Property	Current v1	Current v2
Serving file	<code>model/orientation_model.onnx</code>	<code>model/orientation_model_v2.onnx</code>
Backbone	ResNet-18	EfficientNet-B0
Input	224 × 224	384 × 384
Actual exported preprocessing	Letterbox, ImageNet RGB normalization	Letterbox, ImageNet RGB normalization
ONNX file size	44,704,998 bytes, about 42.63 MiB	16,040,936 bytes, about 15.30 MiB
Recorded training corpus	Same universal manifest	Same universal manifest
Best validation accuracy	98.7261%, epoch 15	98.6465%, epoch 14

Property	Current v1	Current v2
Validation errors, 2,512 rotations	32	34
API route selection	hybrid	pure

Both models are real trained artifacts, not the smoke-test model. Both ONNX metadata maps identify manifest SHA-256 `183047f7e092f1f855f1fc4821498b34f266a31d9480a731136040a5f976040e`, matching the current universal manifest and both saved training configurations. The validation difference is only two predictions and does not establish superiority.

These are different architectures solving the same four-class task. Hybrid is an inference policy, not a separately trained hybrid neural model. Both training runs use the same conventional supervised orientation objective; neither trains OCR jointly with the CNN. The API currently couples backbone/version selection to verification policy, obscuring which component causes a difference. Both current models use letterboxing; describing the deployed v1 as stretched is stale.

The hybrid gate only runs when the two most probable classes are 0 and 180 and either the top score is below 0.90 or their probability gap is below 0.25. It cannot correct 90/270 ambiguity or confident errors that do not pass the gate. The verifier runs text detection plus a text-line direction classifier; it does not transcribe/read words. It accepts a 60% majority of votes above 0.7, with no minimum number of valid lines: one line can override the CNN. These are uncalibrated heuristics.

`RapidOCR()` in installed version 1.4.4 initializes detector, classifier, and recognizer, although this application never uses recognition. It also has its own ONNX thread settings, separate from `APP_INFERENCE_THREADS`. Loading both CNNs and a full OCR engine has a memory/cold-start cost even if most requests do not need all of them. The initialization lock does not protect shared verifier inference; the installed detector mutates `self.preprocess_op` during calls. Use explicit concurrency control or independent process-owned instances.

Recommended product design: one selected model, an explicit uncertain/review outcome, and an optional verifier controlled independently of the model version. Keep alternative architectures for experiments or release rollback. Retain OCR in production only if a paired evaluation demonstrates useful net error reduction on the intended documents, within latency and memory budgets. If deployed, apply verification in a canonical candidate orientation so opposite-direction uncertainty can also be checked after 90/270 candidates; validate this policy separately.

If a future downstream workflow already performs OCR, reuse its detector/direction evidence instead of duplicating that work. For this orientation-only API, loading a full recognizer is unnecessary. A dedicated four-way orientation classifier is a more direct default than a general vision-language model; a text-direction verifier supplies complementary evidence when the global page layout is misleading.

A useful external baseline is PaddleOCR's dedicated four-way [PP-LCNet document orientation model](#). It addresses the same task directly. Its published benchmark is not comparable to this project's test score; run it on the same corrected, held-out documents before replacing anything. A larger ResNet or transformer is not justified by current evidence. Static INT8 calibration is a later CPU optimization experiment, with accuracy revalidation; [ONNX Runtime documents calibration and possible accuracy loss](#).

Measured comparison of the actual serving weights

I evaluated all 539 existing held-out pages at all four rotations, 2,156 examples, through the production ONNX classifier. For each backbone, I compared the identical weights with and without the existing hybrid policy. The v2-plus-OCR row is an experimental combination supported by the classifier, not the current API's default **hybrid** route. No thresholds were tuned during this comparison.

Pipeline	Accuracy against current test labels	Errors / 2,156	Real-source-only accuracy, 1,652 examples	Incorrect rotations of 539 upright pages
ResNet-18 alone	96.94%	66	96.00%	25
ResNet-18 + conditional OCR	97.77%	48	97.09%	13
EfficientNet-B0 alone	97.68%	50	96.97%	8
EfficientNet-B0 + conditional OCR	97.96%	44	97.34%	6

All 504 synthetic test rotations were classified correctly by both models. The lower real-source scores matter more for deployment. In the paired pure comparison, v2 got 35 examples right that v1 missed; v1 got 19 right that v2 missed; both missed 31. Thus neither dominates on every page. Four rotations from a page are correlated, and the label/template limitations below prevent interpreting these numbers as a universal production accuracy claim.

For v1, OCR was attempted on 51 examples (2.37%): 18 mistakes were fixed, zero correct predictions were spoiled, and two additional overrides changed one wrong answer to another wrong answer. For v2, OCR ran on 16 examples (0.74%): six mistakes were fixed and zero correct predictions were spoiled. This is positive evidence that verification adds value; it is not evidence that OCR can never harm on other inputs.

The pure v1 and v2 classifiers still made 26 and 22 errors respectively with confidence at least 0.90. One v1 error had confidence 0.999917. Both selected a 180-degree correction for blank white images at three different aspect ratios. A confidence threshold without blank/ambiguity handling and validation is insufficient.

An isolated, sequential benchmark with only the selected CNN loaded used this machine's Intel Core i5-7400, two inference threads, ONNX Runtime 1.30.0, ten warm-ups, and 50 runs on the same blank 1,240 × 1,754 RGB input:

CNN	Median	p95	Maximum
ResNet-18 / 224	43.14 ms	50.82 ms	54.85 ms
EfficientNet-B0 / 384	65.50 ms	70.05 ms	80.78 ms

This measures resize/normalization/ONNX only, excluding decode, PDF rendering, HTTP, OCR, and output encoding. The image is a timing fixture, not a quality example. V2 has a smaller weight file but is slower here because model structure and higher resolution also affect computation; input pixel count is about 2.94 times larger. These are local Windows results, not AWS estimates.

During the full mixed evaluation, latencies were much more variable: v1 hybrid p99 was approximately 790 ms and v2 hybrid p99 approximately 535 ms, with maxima of 1.94 and 4.08 seconds. Median latency among the OCR-triggered examples was about 761 ms for v1 and 1,150 ms for v2, including that example's CNN call. Different triggered-page sets, lazy initialization, additional runtime sessions, and shared-machine conditions make those OCR figures indicative, not a controlled comparison of verifier speed. The mixed-run metrics are retained instead of substituting the cleaner microbenchmark for actual observed tails.

Model recommendation: use EfficientNet-B0 as the current default candidate, with an independent optional verifier and an abstain/review outcome. It has fewer current-label errors, substantially fewer false corrections of upright pages, and a smaller artifact at a modest isolated CPU latency increase. Keep ResNet-18 as the lower-latency baseline/rollback candidate. V1+OCR remains a valid latency/quality tradeoff, and the difference between the two hybrid totals is only four predictions. Final selection should follow corrected labels, representative document sampling, and AWS end-to-end measurements.

Results: [output/project_review_2026_09_22/model_audit.json](#), [additional_metrics.json](#), [v1_predictions.json](#), [v2_predictions.json](#), and the two [*_isolated_benchmark.json](#) files. The audit script records manifest/model hashes and can reproduce the evaluation locally. ONNX SHA-256 values are [cd3a6d77fa1a8882e12e5adc40729f0b98447f4b3026b9622e66ea6e1302b449](#) for v1 and [d4ae8d22c269753bd324ef45066f8fffb67daafe0a3798fd914eff550f4161060](#) for v2.

What was actually used for training

The current universal manifest contains 4,099 rows and 4,093 unique image hashes. Six duplicate DocLayNet training rows are removed by the loader. There is no exact-hash or recorded-group leakage across its splits. These checks do not detect related templates, re-encodings, or near-duplicates.

Source	Unique training pages	Validation pages	Test pages	Unique total
DocLayNet	1,744	375	375	2,494
CORD receipts	175	37	38	250
FUNSD forms	110	39	0	149
Synthetic Cyrillic	236	34	30	300
Synthetic Arabic	211	49	40	300
Synthetic Indic	218	49	33	300
Synthetic CJK	232	45	23	300
Total	2,926	628	539	4,093

Four generated rotations produce 11,704 training, 2,512 validation, and 2,156 test examples. Four rotations of one page are correlated examples, not four independent documents. The test split has 234 recorded groups, and synthetic grouping further overstates independent template diversity.

The older [data/v2_1000_100](#) manifest has 1,100 rows, 1,097 unique pages, and unique splits of 767/165/165. Its EfficientNet run peaks at 96.3636% validation accuracy. The separate [manifest.reviewed.jsonl](#) contains only 341 training rows, one excluded, and no validation/test split; the saved run points to the original manifest, not this reviewed file. The original [output/](#) run has a 5,000-page configuration but no matching

preserved page manifest, so its data membership cannot be reconstructed from those artifacts alone. Older documentation's 95.99% figure also does not match its current history, whose best validation accuracy is 97.8333%.

DocLayNet is a sensible source of business, legal, scientific, and technical layouts. The full release has 80,863 pages; the project uses a small subset. CORD contributes real photographed receipts; its public release has 1,000 examples with official 800/100/100 splits. FUNSD contributes noisy forms, but its 50-page official test split was not imported here. These are source-dataset facts, not claims about the number of pages used in this project. See the [DocLayNet card](#), [CORD authors](#), and [FUNSD card](#).

The most important data issues are:

1. **Synthetic diversity is overstated.** The 1,200 generated pages are 29.3% of the unique corpus but use only four text templates per script, 16 total. All share a small family of layouts: text near the top, large blank lower regions, occasional borders and seals. Page dimensions, backgrounds, and stamps vary, but content diversity is low. Each generated page gets its own `group_id`, so the split does not hold out template families. A high synthetic test score cannot establish generalization to unfamiliar real multilingual documents.
2. **Multilingual labels are not audited language coverage.** The generator's Cyrillic content is Russian, Arabic content is Arabic, Indic content is Hindi/Devanagari, and CJK content is Chinese/Japanese. It does not supply the full list of languages advertised in its docstring, such as Korean, Bengali, Urdu, or Farsi. CORD is stamped `ko-id-en` uniformly and DocLayNet `en` uniformly; these are assumptions rather than per-page language annotations.
3. **Rendering needs repair.** The inspected synthetic Arabic page visibly contains unjoined letters. The local Pillow build reports no RAQM support, and the generator has no explicit bidi/shaping configuration. Validate Arabic and Indic rendering with script-aware tooling/readers, ensure fonts actually contain the glyphs, and fail rather than silently falling back to a default font. Fixed Windows font paths make generation non-portable. Some long titles can also extend beyond the page width.
4. **The dominant source is already geometrically distorted.** All 2,500 DocLayNet rows contain square $1,024 \times 1,024$ images; an inspected page has stretched glyphs. Letterboxing a square input cannot restore original proportions. Real PDF previews preserve their page ratio. Render the corresponding source PDFs, or recover original geometry using validated source metadata, before applying the shared aspect-preserving pipeline. Re-evaluate/retrain if changing this input distribution.
5. **Verified does not mean manually verified.** Both preparers automatically set remote pages to `upright_verified: true` and assume correction zero. The universal manifest has no exclusions or nonzero corrections. Layout datasets can contain blank, sideways, decorative, or mixed-direction pages. `mark_downloaded_pages_verified` can even change an intentionally unverified row back to true on preparation. Record `verification_method`, reviewer, and audit version; preserve manual review decisions.
6. **Sampling has left important categories out of splits.** The manifest has zero patent training pages, zero laws/regulations validation pages, and zero manual test pages. There are 54 patent validation pages and 67 patent test pages despite no patent training pages. The stream uses a shuffle buffer of 64 followed by taking a prefix, which is not uniform sampling of the full source. Preserve official splits but sample deliberately across documents/categories throughout each source split. Patents account for 37 of the 66 v1 pure test errors, so this is a practical failure mode rather than a theoretical concern.
7. **Universal preparation is not reproducible from arguments alone.** Its synthetic generation and split assignment have no random seed, remote revisions are not pinned, remote failures are swallowed, and

it writes a success message even when sources fail. The standard preparer has revision tracking and resumability, so consolidate around that implementation.

8. **The final evaluation lacks real multilingual forms and target documents.** There are no FUNSD test pages, only 38 receipt test pages, and all explicit non-Latin coverage is generated. Real certificates, IDs, long receipts, smartphone photos, rotated table pages, handwriting, mixed scripts, blur, shadows, perspective distortion, and ambiguous/blank pages need their own evaluation slices if they are in scope.

The current size is enough to develop a promising four-class baseline. It is insufficient evidence for broad 'universal document' reliability. First audit the validation/test labels and add a frozen target-domain test set; then expand real training data by the errors found. A practical next collection might be 500–1,000 independently grouped real target pages for evaluation and several thousand diverse real pages for training, adjusted to the required error rate and domains. These are planning ranges, not a guarantee. For context, even zero failures among 1,000 independent examples only gives an approximate 95% upper error bound of 0.3%; correlated page rotations do not provide 4× the independent evidence.

Use a separate calibration split or validation subset for confidence/verification thresholds. Report performance per source, script, document family, orientation, and acquisition type; also report the rate of incorrectly rotating already-upright pages, abstention coverage, and accuracy on automatically accepted pages. Once this test split informs another design choice, reserve a fresh final test set.

Release-critical application findings

Priority	Finding and evidence	Recommended change
High	<code>app/api/routes.py:167</code> runs whole PDF processing calls in a shared thread pool. Simultaneous requests can execute PyMuPDF in multiple threads.	Use bounded process workers with process-owned documents, or serialize PDF work per process. PyMuPDF explicitly warns that multithreaded use may behave incorrectly or crash; see its multiprocessing documentation .
High	No authentication, rate limits, per-user quotas, or global admission control protect expensive inference. Any caller can choose configured Gemini mode. Wildcard CORS with credentials is also enabled.	Protect the deployment with authentication and rate/concurrency limits; restrict origins. Give Gemini an explicit authorization and spending policy. CORS alone does not control API access.
High	The 10 MiB check occurs after multipart parsing. There is no application-specific decoded-pixel cap or processing deadline. PDF page count and bounded previews do not cap PDF parsing/decompression cost.	Enforce request-body limits before multipart ingestion, explicit image dimensions/pixels, bounded queues, timeouts, and worker memory limits. Pillow's general decompression-bomb protection is not a task memory budget.
High	<code>load_image</code> , rotation, and image encoding run on the async event loop (<code>routes.py:192–215</code>). Only classifier inference is offloaded for images.	Offload the complete image pipeline under bounded capacity, including decode/encode, and load-test health responsiveness alongside large requests.

Priority	Finding and evidence	Recommended change
High	<code>/health</code> always reports <code>status='healthy'</code> and HTTP 200, including when no local models are loaded. Docker only checks this HTTP response.	Separate liveness and readiness; readiness must require the configured production model to load and pass a warm-up inference. Keep remote-service health separate from a mere configured key.
High	Two-frame TIFF input was reproduced returning one frame. <code>load_image</code> and <code>image_to_bytes</code> operate only on the first image.	Reject multipage TIFF/animated inputs explicitly until supported, or process and preserve all frames. Do not silently discard pages.
High	Local modes always choose a class and may rotate ambiguous pages. Softmax is not a calibrated correctness probability.	Introduce a consistent 'uncertain / unchanged / review needed' contract and a validated abstention policy, including blank-page detection. Measure false corrections.
High	Local validation used Python 3.12.5, Pillow 12.3.0, and NumPy 2.5.2; Docker specifies Python 3.11 and requirements constrain Pillow <11 and NumPy <2.	Choose a supported tested dependency set, lock direct/transitive versions, and run tests and model smoke checks inside the exact Linux release image.

The TIFF issue was reproduced using two colored frames without involving model predictions. A separate format probe also reproduced `OSError: cannot write mode CMYK as PNG`: the output format follows the filename while the decoded input mode may differ. Normalize supported pixel modes deliberately, validate content/extension expectations, and return a controlled client error for unsupported conversions. Transparent images should be composited onto a documented background for classification instead of dropping alpha blindly.

Quarter-turn pixel transposition is lossless, but JPEG and WebP output is re-encoded with quality 95, so the complete operation is not lossless. ICC/EXIF and other metadata are not explicitly preserved. PDF text/vector preservation is a strength, but full rewrites are not byte-preserving and should not be advertised as preserving digital signatures. Add tests for any signature, form, annotation, accessibility, metadata, or archival guarantees that the product intends to make.

Gemini's role

The repository now has a third, remote pipeline. It may be useful for a research/demo comparison or an explicitly requested fallback on a small subset, but it should not be the production default without measured quality, cost, privacy, and latency evidence. No live Gemini requests were made during this review.

`docs/GENAI.md` already records a prior live 90/270 correction failure and a provider overload failure; its mocked tests do not establish model accuracy.

`app/core/config.py:18–19` declares `gemini_model` twice; Python keeps the second default, while Compose and `.env.example` specify the first. Consolidate this setting and verify the chosen model's availability for the deployment account. The local health probe reported `inference_mode=genai`, so local configuration differs from the repository's nominal pure default. The parser validates angle and uncertainty but not the requested `title_border`; either remove that extra field or validate any consistency rule you

intend to use. Header location should be a supporting cue, since some documents do not have a meaningful top header.

PDF pages invoke Gemini sequentially. The 60-second network timeout is per operation, not a whole-document deadline; a ten-page request can run for many minutes. An uncertain page makes the entire request fail, and retrying repeats already billed page calls. Use background jobs, explicit partial/uncertain-page status, bounded retries, idempotency/caching with an appropriate data-retention policy, and a total deadline if supporting these requests in production. Do not hide uncertainty by silently invoking paid/external processing.

The UI discloses that Gemini is external; make the actual transmission of page previews and retention policy clear for document owners. A key stored only on the server prevents key exposure but does not prevent unauthorized callers spending its quota. Review the configured account's data-use terms; [Google's logging policy](#) distinguishes private project logs from data explicitly shared for improvement.

Project structure, maintainability, and build integrity

The existing `app/`, `training/`, `tests/`, and `docs/` split is appropriate. Keep the application a modular monolith. Useful targeted cleanup:

- Split `training/dataset.py` into legacy loading, manifest validation, augmentations, and sampling. Retire obsolete public commands instead of keeping parallel preparation paths with different guarantees.
- Give classifiers a small typed protocol and make pipeline policy independent of architecture. Move binary-response/header construction and processing orchestration out of the route. Store services on application state or inject them rather than a module-global singleton.
- Make the evaluator accept both model versions and production ONNX pipelines. It currently rejects v1 and only evaluates the PyTorch v2 classifier. Retain prediction-level outputs and artifact/data hashes for paired comparisons and error analysis.
- Add config validation that forbids v2/224 at training time, since serving rejects that combination. Include model version and all meaningful semantic settings in resume compatibility checks. Currently `model_version` is not checked on resume.
- Separate runtime, training, and test dependencies. Tests currently share a dev requirements file containing the full training stack. Pin the Python version and lock versions for repeatable builds. `pip check` passing only proves installed-package dependency consistency; it does not prove compliance with the project's requirements ranges.
- No `.github` CI or infrastructure/deployment definitions were found. Add Linux lint/tests, image build, artifact injection, readiness/missing-model checks, a small real-model integration set, and container/dependency vulnerability scanning. Add concurrency, TIFF, pixel-limit, and timeout regressions for the fixes above.
- Ruff reports 170 findings on the reviewed working tree; most are line lengths, import ordering, or non-security-sensitive RNG warnings. Resolve actionable correctness/import issues and configure reasonable exceptions for intentional training randomness. Do not present all lint findings as security vulnerabilities.
- `.dockerignore` omits `.venv/`, `data/`, and `output/`. Those directories occupy 5,389.9 MiB, 1,740.6 MiB, and 415.3 MiB respectively in this checkout, about 7.37 GiB combined. Exclude them to bound build-context scanning/transfer. The Dockerfile's explicit `COPY` statements do not put them in the final image, so this is a build-context issue rather than a claim that the image contains the datasets.

- Model binaries are correctly ignored by Git, but no release artifact fetch/verification step replaces them in a clean checkout. Docker can build with only `model/.gitkeep`, and the current health check still succeeds. Publish immutable model artifacts with checksums and inject them deliberately before image creation or during validated startup.
- Compose's local model bind mount is a development convention and can override the baked artifact. Use immutable image/model versions in production with a known rollback path.
- Both training PowerShell scripts use an unqualified `python.exe`, do not set their working directory, and do not explicitly check `$LASTEXITCODE` after native commands. `$ErrorActionPreference='Stop'` alone does not make native nonzero exits fail in Windows PowerShell. Use the project interpreter, stable working directory, and explicit exit checks; avoid unconditional success messages.
- Documentation is materially stale: README/MODEL_V2 still say v2 needs training despite current weights; DATASETS includes commands that the current CLI no longer supports; `pyproject.toml`, schema examples, and app settings disagree on version. `docs/GENAI.md` describes entering extra context in the browser, but there is no corresponding prompt field. Add an artifact-specific model card and a deployment runbook.
- Logs include filenames, which may themselves contain personal information; make that a deliberate retention/redaction decision. Add request duration, model/artifact ID, queue time, page count, outcome, OCR call/override/error counts, peak memory, and external-cost metrics. Request IDs should also appear on failures. Current logs are formatted text rather than JSON structured records.

Training efficiency is reasonable at this scale, but both actual runs used `workers=0`, and each base image is opened/decoded again for each of its four rotations. Profile GPU utilization and data-loading time before expanding the corpus; tune a small number of loader workers and a bounded image cache if decoding is the bottleneck. Do not cache the entire high-resolution corpus blindly. The universal generator currently returns all 1,200 full-resolution synthetic images in one list; yield images instead to bound preparation memory. The standard preparer atomically rewrites the entire growing manifest after each page, which scales quadratically in output text; use periodic durable checkpoints or a recoverable append strategy for larger corpora. Source reweighting exists but both saved runs use `source_weights=null`, so the real training mixture is the raw page mixture above.

The browser UI is serviceable for demonstration. It renders up to ten pages before submitting a PDF, unnecessarily delaying the API call, and uses fixed scale rather than a pixel cap. It never revokes object URLs or explicitly destroys PDF.js documents. It depends on external CDN scripts; self-host reviewed versions for reliable/private deployment and remove the unused ZIP response path/JSZip dependency. Add upload-size checks, cancellation, per-page uncertainty/rotation metadata, keyboard/accessibility checks, and clear differentiation between service errors and preview errors. The displayed timer includes original preview work and excludes some output rendering, so it is not an inference benchmark. PDF headers expose only first-page angle and aggregate decision counts, insufficient for auditing each correction.

AWS deployment approach

For the current small CPU service, my starting choice is an ECR image deployed to ECS Fargate behind an HTTPS Application Load Balancer. Use a small CPU task for staging, initially around 2 vCPU / 4 GiB, then right-size from measured peak memory and end-to-end latency. That size is a starting experiment, not a measured requirement. Limit in-flight work per process to match CPU and memory, load only the selected model, and keep PDF processing in the safe process/serialization boundary described above. There is no demonstrated need for GPU serving or Kubernetes.

Use a dedicated readiness endpoint in both the target-group and task health configuration. ECS does not automatically monitor a Dockerfile health check unless it is specified in the task definition; see [AWS container health checks](#). Inject credentials through [Secrets Manager or SSM integration](#), with scoped IAM roles. Send metrics/logs to CloudWatch, configure alarms and budgets, and retain the previous image/model digest for rollback. Use reproducible infrastructure definitions rather than undocumented console state. A second service task is a production availability choice after validating resource usage, not a prerequisite for a private demo.

For a short-request private demo, App Runner is an alternative with fewer infrastructure components, but it has a [120-second total HTTP request limit](#), making the current sequential Gemini PDFs a poor fit. ALB also has an idle timeout that must agree with application behavior; increasing it is not a substitute for job handling. For long PDFs, use a job endpoint, private S3 objects with expiration, SQS, and bounded workers, returning a job ID immediately. That architecture is optional until the workload requires it. Benchmark complete requests on the chosen AWS hardware before selecting instance size, concurrency, or SLA.

Resolve dataset/package licensing as part of release ownership: DocLayNet publishes CDLA-Permissive-1.0, CORD CC BY 4.0, and the exact FUNSD distribution needs recorded provenance/terms. Also decide how this application's distribution/service will satisfy [PyMuPDF's AGPL or commercial licensing](#). Add a project license and third-party notices where appropriate. These are release decisions; this review does not establish legal compliance.

Suggested order of work

1. Fix TIFF truncation, readiness, PDF concurrency, full image offloading, admission/body/pixel/time limits, and public access/cost controls.
2. Freeze dependencies and model artifacts; build and test the exact Linux image. Add CI and minimal infrastructure/rollback definitions.
3. Audit real validation/test pages, fix synthetic script rendering and template grouping, restore page geometry, and import missing held-out target documents.
4. Compare one backbone with and without verification on the same data. Calibrate abstention on validation and choose a production policy using false-correction rate, coverage, p95 latency, and memory.
5. Deploy a protected AWS staging service, exercise realistic single/multipage concurrent traffic, and use those measurements to set limits and capacity. Promote only after explicit quality and operational acceptance criteria pass.

Verification scope and limitations

The existing suite passed: 128 tests in 26.21 seconds, with two dependency deprecation warnings. [pip check](#) passed. Ruff reported 170 findings before adding review artifacts. The current manifest's checksums and recorded split boundaries were validated by the audit loader. TIFF truncation, missing-model health behavior, and the incompatible image-mode conversion were reproduced locally.

Docker was not available on this machine, so the Linux image was not built or executed. No AWS resources were created, no full HTTP load test was run, no live Gemini quality/cost test was made, and no comprehensive dependency/CVE scan was performed. Local sequential timing cannot establish AWS capacity. Existing test labels were not exhaustively manually audited. Numerical model findings below must be interpreted under those limits.